Reviewer Pack — Minimal Number of Invariant Generators for Affine Sheaves an…

Entry 9cde66aeb5ac · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 05:42:07 UTC

Minimal Number of Invariant Generators for Affine Sheaves and Communication Complexity Rank

Entry ID: `9cde66aeb5ac` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 05:42:07 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Sheaf Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For every n-vertex graph G with m edges, the minimum number of invariant generators required to describe the sheaf cohomology of its incidence complex is linearly related to its communication complexity rank, such that min_gen(G) = Θ(rank_comm(G)), where min_gen(G) denotes the smallest number of invariant generators for G's incidence complex and rank_comm(G) is the communication complexity rank of G.

Rationale (proposer's reasoning):

> Sheaf theory provides a framework for studying geometric structures in algebraic geometry, which can be used to encode combinatorial problems. Communication complexity, on the other hand, measures the amount of information that two parties need to exchange in order to compute a function. The conjecture suggests that the inherent structure encoded by sheaves in the incidence complex of a graph is closely related to its communication complexity rank, potentially revealing new insights into both fields.

Taxonomy category: `Sheaf Theory × Communication Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `d4ebde3e19b0100f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Pearson correlation coefficient between min_gen(G) and rank_comm(G) is ≥ 0.8 AND the mean of the absolute differences between min_gen(G) and Θ(rank_comm(G)) across all n ≤ 40 graphs with m edges is ≤ 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "affine sheaves" AND "communication complexity rank" - "incidence complex sheaf cohomology" AND "linear relationship" - "invariant generators sheaf theory" AND "matrix rank communication complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_graph(n, m):
    if n * (n - 1) // 2 < m:
        raise ValueError("Graph size must be a multiple of the degree")

    G = {i: set() for i in range(n)}
    edges_added = 0

    while edges_added < m:
        u, v = random.sample(range(n), 2)
        if u not in G[v]:
            G[u].add(v)
            G[v].add(u)
            edges_added += 1

    return G

def incidence_matrix(G):
    n = len(G)
    m = sum(len(neighbors) for neighbors in G.values()) // 2
    M = [[0] * (n + m) for _ in range(n)]

    for u, neighbors in enumerate(G.items()):
        for v in neighbors[1]:
            M[u][v] += 1

    return M
```

```
def gaussian_elimination(M):
    n = len(M)
    m = len(M[0])
    rank = 0
    pivot_col = 0

    for i in range(n):
        if pivot_col >= m:
            break

        max_row = i
        for r in range(i + 1, n):
            if abs(M[r][pivot_col]) > abs(M[max_row][pivot_col]):
                max_row = r

        M[i], M[max_row] = M[max_row], M[i]

        if M[i][pivot_col] == 0:
            pivot_col += 1
            continue

        rank += 1
        for j in range(m):
            M[i][j] /= M[i][pivot_col]

        for r in range(n):
            if r != i and M[r][pivot_col] != 0:
                factor = -M[r][pivot_col]
                for j in range(m):
                    M[r][j] += factor * M[i][j]

        pivot_col += 1

    return rank

def min_invariant_generators(G):
    n = len(G)
    M = incidence_matrix(G)

    # Add identity matrix to the right of M
    for i in range(n):
        M[i].extend([0] * (n - i))
        M[i][i + n] = 1

    rank = gaussian_elimination(M)
    return rank

def communication_complexity_rank(G):
    n = len(G)
    m = sum(len(neighbors) for neighbors in G.values()) // 2
    rank = 0

    # Compute the adjacency matrix
    A = [[0] * n for _ in range(n)]
    for u, neighbors in enumerate(G.items()):
```

```
            for v in neighbors[1]:
                A[u][v] = 1

        # Perform Gaussian elimination on A
        rank_A = gaussian_elimination(A)

        return rank_A

    def run_trial(seed: int) -> dict:
        random.seed(seed)
        n_values = [5, 10, 15, 20, 30, 40]
        results = []

        for n in n_values:
            m = random.randint(n - 1, n * (n - 1) // 2)
            G = generate_graph(n, m)

            min_gen = min_invariant_generators(G)
            rank_comm = communication_complexity_rank(G)

            results.append({
                "n": n,
                "m": m,
                "min_gen": min_gen,
                "rank_comm": rank_comm
            })

        if not results:
            return {
                "metric_name": "Pearson correlation coefficient",
                "metric_value": None,
                "instances_tested": 0,
                "n_max": 0,
                "conjecture_holds": False,
                "counterexample": "No valid graphs generated"
            }

        min_gen_values = [r["min_gen"] for r in results]
        rank_comm_values = [r["rank_comm"] for r in results]

        mean_min_gen = sum(min_gen_values) / len(min_gen_values)
        mean_rank_comm = sum(rank_comm_values) / len(rank_comm_values)

        abs_diffs = [abs(m - n) for m, n in zip(min_gen_values, rank_comm_values)]
        mean_abs_diff = sum(abs_diffs) / len(abs_diffs)

        correlation_coefficient = 0
        if mean_rank_comm != 0:
            covariance = sum((m - mean_min_gen) * (n - mean_rank_comm) for m, n in zip(min_gen_values,
            variance_min_gen = sum((m - mean_min_gen) ** 2 for m in min_gen_values)
            variance_rank_comm = sum((n - mean_rank_comm) ** 2 for n in rank_comm_values)
            correlation_coefficient = covariance / (math.sqrt(variance_min_gen) * math.sqrt(variance_r

        return {
            "metric_name": "Pearson correlation coefficient",
            "metric_value": correlation_coefficient,
```

```
        "instances_tested": len(results),
        "n_max": max(r["n"] for r in results),
        "conjecture_holds": correlation_coefficient >= 0.8 and mean_abs_diff <= 3,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
        results.append(trial_result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(re
        support_fraction = len([r for r in results if r["conjecture_holds"]]) / len(results)
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67e016c7.py", line 167, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67e016c7.py", line 117, in run_trial
    min_gen = min_invariant_generators(G)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67e016c7.py", line 89, in min_invariant
    rank = gaussian_elimination(M)
           ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67e016c7.py", line 68, in gaussian_elim
    M[i][j] /= M[i][pivot_col]
    ~~~~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means it was unable to compute the required metrics for the conjecture. | next: Investigate and fix the crash in the test code to allow for the computation of min_gen(G) and rank_comm(G), then re-run the test.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15321
2	preregistration	ollama_remote	glm4:latest	0	0	10410
3	novelty	ollama_remote	glm4:latest	0	0	8992
4	novelty	ollama_remote	glm4:latest	0	0	9179
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	43774
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11396
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15110
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17322
9	judge	ollama_remote	glm4:latest	0	0	15449

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 146952 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/9cde66aeb5ac.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9cde66aeb5ac.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/9cde66aeb5ac.tar.gz` (if generated)